UNIVERSIDADE CATÓLICA DE SANTOS

CIÊNCIA DA COMPUTAÇÃO

COMPUTAÇÃO GRÁFICA E PROCESSAMENTO DE IMAGENS

Revisão Bibliográfica:

Bloom e Efeitos de Pós-Processamento em Tempo Real

Integrantes do Grupo:

Arthur Iwankiu Castro (RA: 2822116)

Enzo da Silva Passos (RA: 7047045)

Francklin Roberto dos Santos Junior (RA: 9125435)

Kayque de Jesus Levy Rodrigues (RA: 6593272)

Miguel Nepomuceno Gil (RA: 5196718)

Vitor Oliveira Christovam (RA: 2362780)

Santos – SP

2026

Resumo

Este trabalho apresenta uma pesquisa teórica e bibliográfica sobre a fundamentação física, a modelagem matemática e a implementação prática do efeito de *Bloom* e dos pipelines de pós-processamento em computação gráfica em tempo real. Discutem-se os princípios ópticos e fisiológicos da dispersão luminosa intraocular e em lentes fotográficas, a extração analítica de luminância com transição contínua (*soft knee*), a decomposição de convoluções Gaussianas e piramidais, bem como a compressão por mapeamento tonal (*tone mapping*). Detalha-se a arquitetura de implementação sob a API de baixo nível Vulkan, cobrindo o uso de *Fullscreen Triangles*, transições de layout de imagem e barreiras de memória explícitas. Por fim, analisam-se aplicações em motores gráficos e realizam-se estudos de caso analíticos com capturas de tela obtidas durante o gameplay em *Portal with RTX* e *Cyberpunk 2077*. Na segunda etapa do trabalho, apresenta-se o desenho técnico da arquitetura do pipeline Vulkan da técnica, com seus *render passes*, *descriptor sets*, *buffers*, *shaders*, barreiras de memória e transições de *layout* de imagem. Na terceira etapa, apresenta-se o pseudocódigo, em sintaxe GLSL, dos *shaders* desse pipeline, com a explicação de suas entradas, saídas, cálculos, *loops*, condições e amostragens de textura.

1 Introdução

A renderização e exibição fidedigna de fenômenos luminosos em computação gráfica em tempo real enfrenta uma severa limitação de natureza física e tecnológica: a faixa dinâmica restrita (*Low Dynamic Range* – LDR) dos monitores de exibição convencionais [1]. Enquanto tais dispositivos operam normalizados no intervalo $[0, 1]$, fontes luminosas no mundo físico (como arcos elétricos, filamentos incandescentes e o Sol) emitem radiação com ordens de magnitude substancialmente superiores às de superfícies puramente difusas circundantes [1].

Para mitigar a perda de contraste perceptivo sem saturar indiscriminadamente todo o sinal cromático da cena, a computação gráfica moderna emprega filtros espaciais em espaço de tela (*screen-space*) através de passes de pós-processamento [5]. Dentre esses, o *Bloom* desempenha um papel essencial ao simular computacionalmente os efeitos ópticos de difração e dispersão que ocorrem naturalmente no humor vítreo e no cristalino do olho humano ou nos elementos de lentes de câmeras fotográficas [2, 5].

2 Explicação Teórica da Técnica

2.1 Definição e Objetivo

O *Bloom* (frequentemente tratado na literatura como *glare* ou *light bleeding*) consiste na simulação computacional do espalhamento difuso de fótons quando estes atravessam meios ópticos imperfeitos [2]. Seu objetivo primordial é transmitir ao observador a percepção de intensidade extrema em fontes emissivas ou reflexões especulares, sobrepujando a incapacidade física dos displays de emitir luminância superior ao seu limiar nativo de brilho [5].

2.2 Como a Técnica Funciona

O pipeline clássico de pós-processamento para geração do *Bloom* opera em quatro fases ordenadas:

1. **Bright Pass (Isolamento de Altas Luzes):** Realiza uma varredura sobre o *framebuffer* HDR da cena, isolando e filtrando os fragmentos cuja intensidade de luminância ultrapassa um determinado limiar de corte (*threshold*).

2. **Downsampling e Filtragem Espacial:** Redução piramidal progressiva da resolução da textura isolada combinada à aplicação encadeada de filtros de desfoque espacial (como convoluções Gaussianas ou pirâmides de filtragem), gerando múltiplos níveis de dispersão com raios variados [5].

3. **Upsampling e Composição Aditiva:** Interpolação bilinear progressiva das camadas desfocadas da menor para a maior resolução, acumulando aditivamente as contribuições espalhadas sobre a imagem original da cena.

4. **Tone Mapping (Mapeamento Tonal):** Compressão matemática da imagem HDR combinada para o intervalo $[0, 1]$ aceito pelos dispositivos LDR, preservando detalhes tanto em penumbra quanto em altas claridades [3].

2.3 Problema Resolvido, Vantagens e Limitações

A técnica soluciona a perda de contraste perceptivo em que superfícies brancas puras $(1.0, 1.0, 1.0)$ e fontes ativamente incandescentes seriam representadas com idêntico valor após a rasterização direta.

Vantagens: Apresenta custo computacional reduzido e previsível, operando estritamente em espaço de tela 2D após a resolução da geometria tridimensional.

Limitações: A calibração inadequada de limiar ou do raio de espalhamento pode introduzir embaçamento generalizado (*haze*), saturação excessiva e perda de nitidez estrutural dos modelos 3D.

2.4 Custo Computacional e Aplicação Adequada

O processamento independe da complexidade geométrica, contagem de polígonos ou complexidade de materiais da cena, apresentando custo temporal puramente linear $\mathcal{O}(P)$, onde P é o total de pixels processados. É indispensável para cenas noturnas com iluminação artificial, cenas externas sob luz solar direta e jogos com superfícies altamente reflexivas.

3 Equações Matemáticas Envolvidas

3.1 Luminância Perceptual (Espaço ITU-R BT.709)

A filtragem de fragmentos candentes fundamenta-se na sensibilidade espectral da visão humana, que prioriza a faixa espectral do verde:

$$L = 0.2126\,R + 0.7152\,G + 0.0722\,B \tag{1}$$

Significado das variáveis:

- R, G, B: Canais lineares de cor com precisão de ponto flutuante (*floating-point*).
- L: Luminância escalar perceptual calculada para o fragmento.

Momento de uso e relação com a renderização: Utilizada na etapa preliminar de *Bright Pass* para avaliar quais fragmentos possuem energia suficiente para gerar halos de dispersão.

3.2 Extração com Joelho Suave (*Bright-Pass Soft Knee*)

Para evitar descontinuidades e ruídos de cintilação temporal (*fireflies*) ao longo de bordas móveis, emprega-se uma função de transição contínua com joelho suave:

$$C_{\text{bright}} = C_{\text{cena}} \cdot \frac{\max\left(0, \min(L - T + k, 2k)\right)^2}{4k + \epsilon} \tag{2}$$

Significado das variáveis:

- C_{cena}: Vetor tridimensional de cor HDR original do pixel.
- T: Limiar de corte (*threshold*) estipulado artisticamente.
- k: Fator escalar de abertura da transição suave (*knee*).
- ϵ: Constante infinitesimal para evitar divisões por zero.
- C_{bright}: Vetor de cor filtrado que alimenta as etapas subsequentes de convolução.

Momento de uso e relação com a renderização: Executada no *Fragment Shader* ou *Compute Shader* que gera o primeiro nível de resolução da cadeia de dispersão.

3.3 Função de Espalhamento Pontual de Spencer (*Point Spread Function*)

A base biofísica da dispersão de luz decorrente de micropartículas no olho humano foi formalizada por Spencer et al. [2]:

$$f(\theta) = \frac{k_1}{\theta^3 + \epsilon} + \frac{k_2}{\theta^2} \tag{3}$$

Significado das variáveis:

- θ: Ângulo de desvio angular da linha direta de visada em radianos.
- k_1, k_2: Coeficientes empíricos relacionados à difração no cristalino e à dispersão na córnea.
- ϵ: Termo de amortecimento que evita singularidades numéricas quando $\theta \to 0$.

Momento de uso e relação com a renderização: Constitui o embasamento físico do fenômeno; por ter cálculo integral contínuo custoso, é discretizada mediante múltiplos passos de filtragem espacial [5].

3.4 Kernel Gaussiano Separável Unidimensional

A discretização em tempo real da PSF efetua-se decompondo o filtro bidimensional em dois passos ortogonais (horizontal e vertical), barateando o custo por pixel de $\mathcal{O}(K^2)$ para $\mathcal{O}(2K)$:

$$G(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{x^2}{2\sigma^2}\right) \tag{4}$$

Significado das variáveis:

- x: Deslocamento discreto em coordenadas de *texels* a partir do centro da convolução.
- σ: Desvio padrão da distribuição Gaussiana, definidor do raio de dispersão luminosa.

Momento de uso e relação com a renderização: Aplicada a cada nível da cadeia de amostragem regressiva (*downsampling*) e progressiva (*upsampling*) para expandir o halo de brilho.

3.5 Mapeamento Tonal Fotográfico de Reinhard

Para recombinar o brilho acumulado à imagem da cena e comprimir a faixa dinâmica sem achatamento abrupto, emprega-se a formulação de Reinhard et al. [3]:

$$C_{\text{final}} = \frac{C_{\text{total}}}{1 + C_{\text{total}}}, \quad \text{onde } C_{\text{total}} = C_{\text{cena}} + \alpha C_{\text{bloom}} \tag{5}$$

Significado das variáveis:

- C_{cena}: Cor HDR original da cena renderizada.
- C_{bloom}: Contribuição de cor acumulada pela pirâmide de dispersão.
- α: Parâmetro multiplicativo de calibração da intensidade do *Bloom*.
- C_{final}: Sinal cromático mapeado para o intervalo normalizado $[0, 1]$.

Momento de uso e relação com a renderização: Etapa final do pipeline de pós-processamento, antecedendo o envio do *framebuffer* ao *Swapchain*.

4 Relação da Técnica com a API Vulkan

4.1 Shaders e Pipelines Gráficos

A arquitetura do *Bloom* no Vulkan implementa-se de forma enxuta sem necessidade de alocação de malhas ou *Vertex Buffers*:

- **Fullscreen Triangle:** O *Vertex Shader* gera um triângulo analítico único diretamente a partir do identificador intrínseco `gl_VertexIndex`. O polígono recobre integralmente o espaço de coordenadas de tela (NDC), suprimindo os custos de interpolação duplicada na diagonal característicos de quads tradicionais.
- **Compute Shaders e Memória Compartilhada:** Nos passos de convolução, agrupam-se *threads* em grupos locais (*workgroups* de 16×16). Os dados dos *texels* vizinhos são carregados em memória compartilhada (*shared memory* / LDS), contornando leituras redundantes na VRAM e maximizando a vazão de registradores da GPU.

4.2 Buffers, Attachments e Sincronização

O fluxo de dados transita sequencialmente:
`Cena HDR` → `Bright Pass` → `Mip Chain (Down)` → `Upsample (Tent Filter)` → `Tone Mapping` → `Swapchain`.

O *framebuffer* primário requer formatos em ponto flutuante de 16 ou 32 bits, comumente:

`VK_FORMAT_R16G16B16A16_SFLOAT`

As texturas alocadas para a pirâmide de resolução demandam o uso simultâneo de sinalizadores na estrutura `VkImageCreateInfo`:

`VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT | VK_IMAGE_USAGE_SAMPLED_BIT | VK_IMAGE_USAGE_STORAGE_BIT`

A alternância segura entre etapas de escrita e leitura nos múltiplos níveis é coordenada por barreiras de sincronização explícitas (`vkCmdPipelineBarrier`), que efetuam transições de `VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL` para `VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL`, estabelecendo dependências estritas entre os estágios `VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT` e `VK_PIPELINE_STAGE_FRAGMENT_SHADER_BIT`.

5 Aplicações Práticas

- **NVIDIA RTX Remix:** No pipeline modular de traçado de caminhos em tempo real (*Path Tracing*), o *Bloom* atua subsequentemente ao desruído temporal, atenuando transições de alto contraste e dispersando a energia de emissores de radiação física sobre superfícies vizinhas em espaço de tela [4].

- **Unreal Engine 5:** Integração de desfoque convolucional de abertura física com o sistema de iluminação global dinâmica Lumen, conferindo volume perceptivo e legibilidade visual a materiais emissivos em ambientes de penumbra.

6 Análise da Técnica em Jogos

6.1 Jogo 1: Portal with RTX

- **Nome do jogo e Plataforma:** *Portal with RTX*, executado em computador pessoal com sistema operacional Windows 11.

- **Situação de gameplay analisada:** Câmara de testes durante o disparo e trânsito da esfera de energia de alta densidade (*High Energy Pellet*) próxima às paredes metálicas modulares reflexivas.

- **Momento em que a técnica pode ser observada:** No instante em que o projétil incandescente cruza a sala e se aproxima da superfície metálica vertical da câmara de testes.

- **Elemento visual de aplicação:** Esfera emissora de plasma em movimento no ar e a superfície modular metálica da parede adjacente.

- **Efeito visual produzido:** O projétil pontual de plasma (círculo ciano) gera um halo volumétrico difuso e intenso que ultrapassa a geometria do objeto em direção à visão do jogador; simultaneamente, o reflexo especular refletido nas placas facetadas da parede (círculo vermelho) acumula luminância suficiente para ativar o filtro de *Bright Pass*, transbordando um gradiente de sangramento de luz (*light bleeding*) sobre as costuras escuras dos painéis vizinhos.

- **Benefício da técnica para o jogo:** Transmite a sensação tátil de perigo e calor concentrado da esfera de plasma, evitando que o reflexo especular pareça uma mancha opaca e integrando a iluminação por traçado de caminhos (*Path Tracing*) aos limites de contraste do monitor.

- **Fonte técnica confirmatória:** A formulação de filtros em espaço de tela (*image-space*), isolamento de altas intensidades e aceleração de dispersão luminosa em pipelines HDR descrita por Ignatenko e Barladyan [4].

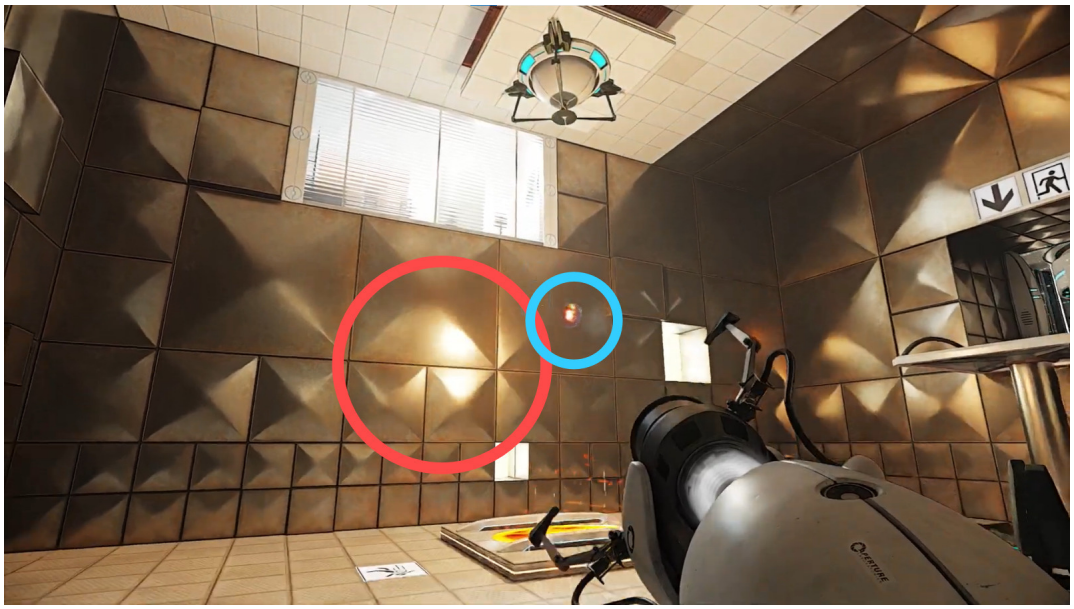


Figura 1: Captura de tela obtida durante o gameplay de *Portal with RTX*. O círculo ciano destaca o projétil de plasma incandescente em trânsito com saturação no núcleo e halo de dispersão; o círculo vermelho evidencia o reflexo especular de alta intensidade na parede metálica com sangramento de luz (*light bleeding*) sobre as junções dos painéis.

6.2 Jogo 2: Cyberpunk 2077

- **Nome do jogo e Plataforma:** *Cyberpunk 2077*, executado em computador pessoal (PC/Windows 11) sobre o motor gráfico REDengine 4.

- **Situação de gameplay analisada:** Exploração noturna a pé no distrito urbano em frente ao estabelecimento comercial *Lizzie's Bar*, em ambiente com alta concentração de fontes emissivas de néon e piso asfáltico molhado com reflexões especulares.

- **Momento em que a técnica pode ser observada:** No momento em que a visão do jogador mira a fachada iluminada com luminosos de néon e as poças de chuva refletoras na via pública.

- **Elemento visual de aplicação:** Painel publicitário emissivo superior (*WET DREAM*), silhuetas figurativas de néon na fachada e reflexos diretos no chão molhado.
- **Efeito visual produzido:** O painel publicitário de alta luminância (demarcado pelo círculo ciano) gera um halo de dispersão volumétrica que transborda os limites da tela. Simultaneamente, as silhuetas de néon (círculo magenta) e as reflexões no piso (círculo vermelho) produzem sangramento cromático suave sobre a dispersão atmosférica ambiente (retângulo amarelo).
- **Benefício da técnica para o jogo:** Essencial para conferir a atmosfera densa e visualmente distópica do cenário urbano futurista, transmitindo a intensidade física das luzes artificiais contra a noite e ampliando a percepção de contraste.
- **Fonte técnica confirmatória:** O modelo de avaliação perceptiva e simulação de dispersão luminosa (*glare*) em imagens HDR detalhado por Čadík et al. [5].



Figura 2: Captura de tela obtida durante a execução do gameplay de *Cyberpunk 2077*. O círculo ciano isola o painel emissivo principal (*WET DREAM*); o círculo magenta demarca a iluminação de néon da fachada; o retângulo amarelo indica a dispersão difusa na névoa ambiente; o círculo vermelho destaca o sangramento de luz na reflexão especular do solo molhado.

7 Arquitetura do Pipeline Vulkan

Esta seção é a segunda entrega do trabalho. A Figura 3 mostra o desenho do pipeline Vulkan que executa o *Bloom*, feito no draw.io. Como exemplo, o grupo considerou uma

tela de 1920 × 1080, imagens no formato `VK_FORMAT_R16G16B16A16_SFLOAT`, citado na Seção 4, e uma pirâmide de cinco níveis, de `bloomMip[0]` a `bloomMip[4]`.

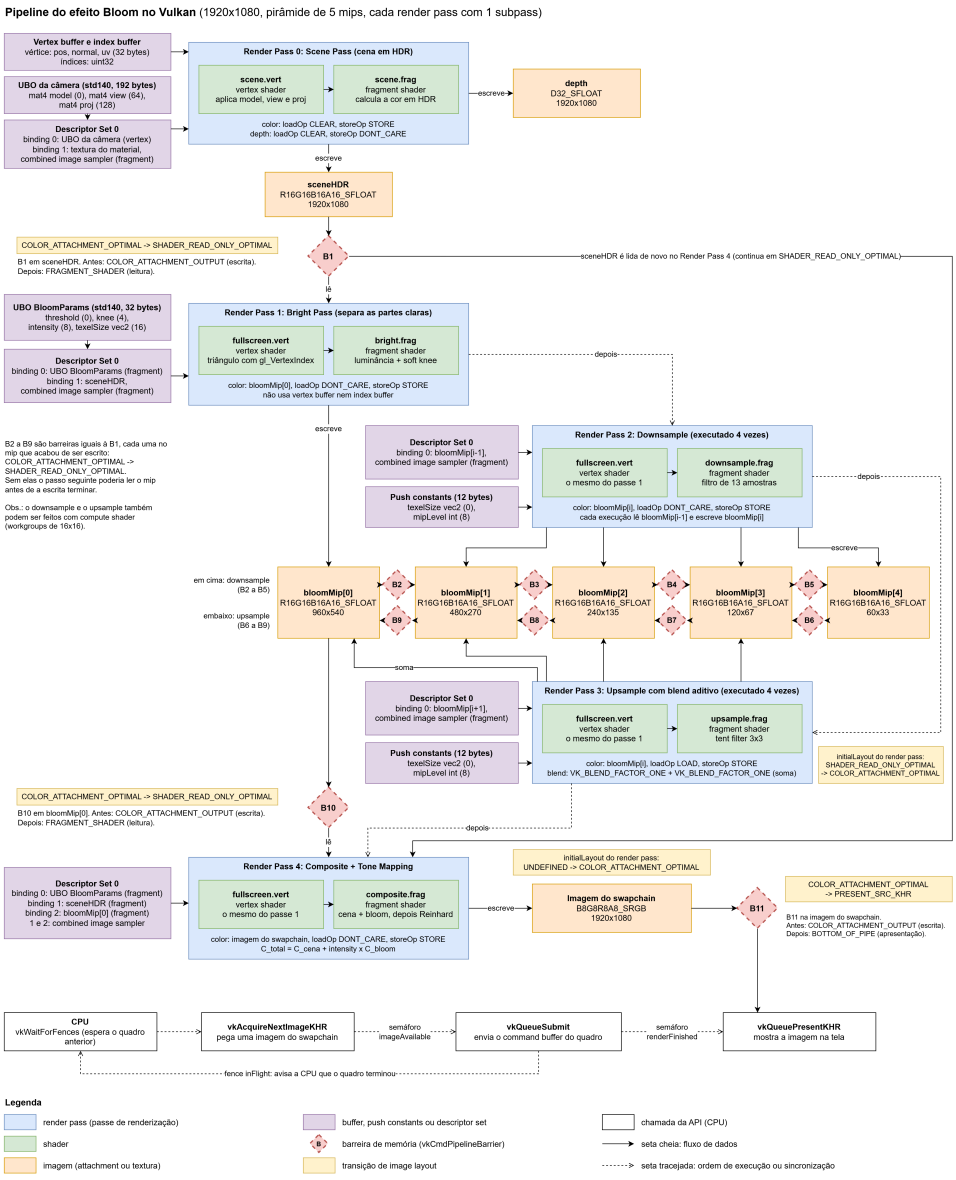


Figura 3: Arquitetura do pipeline Vulkan para o efeito de *Bloom*. Azul: *render pass*. Verde: *shader*. Laranja: imagem. Roxo: *buffer* ou *descriptor set*. Losango vermelho: barreira de memória. Amarelo: transição de *layout*. Seta cheia: dados. Seta tracejada: ordem de execução ou sincronização.

7.1 Fluxo de Dados

O quadro é desenhado em onze passes, gravados em ordem em um único *command buffer*:

1. **Passe 0 (*Scene Pass*):** recebe o *vertex buffer*, o *index buffer*, o UBO da câmera e as texturas. Desenha a cena na `sceneHDR`, usando a `depth` no teste de profundidade.

2. **Passe 1 (*Bright Pass*):** lê a `sceneHDR` e guarda só as partes claras na `bloomMip[0]`, com metade da resolução.
3. **Passes 2 a 5 (*Downsample*):** cada execução lê `bloomMip[i-1]` e escreve `bloomMip[i]`, menor e mais borrado.
4. **Passes 6 a 9 (*Upsample*):** cada execução lê `bloomMip[i+1]` e soma o resultado em `bloomMip[i]`. No fim, a `bloomMip[0]` tem o halo completo.
5. **Passe 10 (*Composite*):** soma a `sceneHDR` com a `bloomMip[0]`, aplica o *tone mapping* e escreve na imagem do *swapchain*, que é mostrada na tela.

Recursos compartilhados. A `sceneHDR` é lida nos passes 1 e 10. Os níveis `bloomMip` são *mip levels* de uma mesma `VkImage`, usados na descida e na subida. O UBO `BloomParams` serve aos passes 1 e 10, e o `fullscreen.vert` e o `VkSampler` são os mesmos em todos os passes de tela cheia.

7.2 Render Passes e Subpasses

São cinco *render passes*, todos com um único *subpass* (Tabela 1).

Tabela 1: *Render passes* do pipeline.

RP	Passes	Attachment	loadOp / storeOp	initialLayout	Resultado
0	0	`sceneHDR` (cor)	`CLEAR / STORE`	`UNDEFINED`	cena em HDR
		`depth` (profundidade)	`CLEAR / DONT_CARE`	`UNDEFINED`	teste de profundidade
1	1	`bloomMip[0]`	`DONT_CARE / STORE`	`UNDEFINED`	partes claras da cena
2	2 a 5	`bloomMip[1]` a `bloomMip[4]`	`DONT_CARE / STORE`	`UNDEFINED`	nível menor e borrado
3	6 a 9	`bloomMip[3]` a `bloomMip[0]`	`LOAD / STORE`	`SHADER_READ_ONLY_OPTIMAL`	soma dos níveis
4	10	imagem do *swapchain*	`DONT_CARE / STORE`	`UNDEFINED`	imagem final

No início do *subpass*, o *render pass* leva o *attachment* do `initialLayout` para `COLOR_ATTACHMENT_OPTIMAL`. O *upsample* usa `LOAD` porque precisa manter o conteúdo anterior para a soma. Os passes ficaram em *render passes* separados porque cada um escreve em uma imagem de tamanho diferente e lê *texels* vizinhos da anterior com um *sampler*, o que um *input attachment* não permite. O `VK_KHR_dynamic_rendering` não foi usado por não ser o modelo visto na disciplina.

7.3 Descriptor Sets e Bindings

Todos os pipelines usam só o *set* 0 (Tabela 2). No *downsample* e no *upsample* há um *descriptor set* para cada nível lido.

Tabela 2: *Descriptor sets* e *bindings* de cada passe.

Passe	Set	Binding	Tipo	Recurso	Estágio
0 (*Scene*)	0	0	`UNIFORM_BUFFER`	UBO da câmera	*vertex*
	0	1	`COMBINED_IMAGE_SAMPLER`	textura do material	*fragment*
1 (*Bright*)	0	0	`UNIFORM_BUFFER`	`BloomParams`	*fragment*
	0	1	`COMBINED_IMAGE_SAMPLER`	`sceneHDR`	*fragment*
2 a 5 (*Downsample*)	0	0	`COMBINED_IMAGE_SAMPLER`	`bloomMip[i-1]`	*fragment*
6 a 9 (*Upsample*)	0	0	`COMBINED_IMAGE_SAMPLER`	`bloomMip[i+1]`	*fragment*
10 (*Composite*)	0	0	`UNIFORM_BUFFER`	`BloomParams`	*fragment*
	0	1	`COMBINED_IMAGE_SAMPLER`	`sceneHDR`	*fragment*
	0	2	`COMBINED_IMAGE_SAMPLER`	`bloomMip[0]`	*fragment*

7.4 Layout e Finalidade dos Buffers

- ***Vertex buffer* e *index buffer*:** geometria da cena, só no passe 0. Cada vértice tem 32 bytes: posição (*offset* 0), normal (12) e coordenada de textura (24). Os índices são `uint32`.
- **UBO da câmera** (`std140`, 192 bytes): matrizes `model` (0), `view` (64) e `proj` (128).
- **UBO** `BloomParams` (`std140`, 32 bytes): `threshold` (0), `knee` (4), `intensity` (8) e `texelSize` (16), que são os parâmetros das Equações 2 e 5.
- ***Push constants*** (12 bytes): `texelSize` (0) e `mipLevel` (8), que mudam a cada execução do *downsample* e do *upsample*.

Os passes 1 a 10 não têm *vertex buffer*: o `fullscreen.vert` gera o triângulo de tela cheia a partir do `gl_VertexIndex`, desenhado com `vkCmdDraw(3, 1, 0, 0)`.

7.5 Shaders Envolvidos

O pipeline usa apenas *vertex* e *fragment shaders*:

- `scene.vert` e `scene.frag`: transformam os vértices e calculam a cor em HDR, que pode passar de 1.0. Nos passes 1 a 10 o *vertex shader* é sempre o `fullscreen.vert`.
- `bright.frag`: luminância (Equação 1) e corte com joelho suave (Equação 2).
- `downsample.frag`: média de 13 amostras que reduz e borra, no papel do filtro da Equação 4.
- `upsample.frag`: *tent filter* 3×3. A soma é feita pelo *blend* aditivo, com `VK_BLEND_FACTOR_ONE` na origem e no destino.
- `composite.frag`: soma a cena com o *Bloom* e aplica o *tone mapping* de Reinhard (Equação 5) [3].

7.6 Barreiras de Memória e Transições de Layout

A GPU pode começar a ler uma imagem antes de a escrita terminar. Além disso, o *attachment* fica em `COLOR_ATTACHMENT_OPTIMAL` e a leitura por *sampler* exige `SHADER_READ_ONLY_OPTIMAL`. Por isso há uma barreira (`vkCmdPipelineBarrier`) sempre que uma imagem escrita em um passe é lida no seguinte (Tabela 3).

Tabela 3: As 11 barreiras de memória do diagrama.

	Imagem	**Escrita → leitura**		**Imagem**	**Escrita → leitura**
B1	`sceneHDR`	passe 0 → passes 1 e 10	B7	`bloomMip[3]`	passe 6 → passe 7
B2	`bloomMip[0]`	passe 1 → passe 2	B8	`bloomMip[2]`	passe 7 → passe 8
B3	`bloomMip[1]`	passe 2 → passe 3	B9	`bloomMip[1]`	passe 8 → passe 9
B4	`bloomMip[2]`	passe 3 → passe 4	B10	`bloomMip[0]`	passe 9 → passe 10
B5	`bloomMip[3]`	passe 4 → passe 5	B11	*swapchain*	passe 10 → apresentação
B6	`bloomMip[4]`	passe 5 → passe 6			

Nas barreiras B1 a B10 os dados são os mesmos: o acesso anterior é uma escrita (`COLOR_ATTACHMENT_WRITE`) no estágio `COLOR_ATTACHMENT_OUTPUT`, o acesso posterior é uma leitura (`SHADER_READ`) no estágio `FRAGMENT_SHADER` e o *layout* muda de `COLOR_ATTACHMENT_OPTIMAL` para `SHADER_READ_ONLY_OPTIMAL`. Na B11 o acesso anterior é o mesmo, depois não há outro acesso da GPU (estágio `BOTTOM_OF_PIPE`) e o *layout* vira `PRESENT_SRC_KHR`, exigido para a apresentação. O motivo é igual em todas: a imagem só pode ser usada depois que a escrita terminou e ela está no *layout* certo.

As outras transições de *layout* são feitas pelos *render passes* com o `initialLayout` e aparecem em amarelo no diagrama. A `sceneHDR` é lida de novo no passe 10 sem outra barreira, porque continua no *layout* de leitura. A sincronização entre CPU e GPU usa os semáforos `imageAvailable` e `renderFinished` e a *fence* `inFlight`, mostrados na parte de baixo do diagrama.

7.7 Justificativa das Decisões

- **Triângulo de tela cheia:** dispensa *vertex buffer* e evita a diagonal que um *quad* teria (Seção 4).
- **Pirâmide de *mips*:** um borrão largo feito de uma vez em resolução cheia precisaria de muitas amostras por pixel (Equação 4). Com a imagem reduzida pela metade a cada nível, um filtro pequeno alcança um raio grande, e a soma dos níveis lembra o espalhamento de Spencer et al. [2].
- **Formato de 16 bits em ponto flutuante:** guarda valores acima de 1.0, que diferenciam uma luz forte de uma superfície branca [1]. Com 8 bits eles seriam cortados.

- ***Fragment shader* em vez de *compute shader*:** a Seção 4 cita que o borrão também pode usar *compute shaders* em *workgroups* de 16×16. O grupo desenhou a versão com *fragment shader* por ser mais simples: todos os passes têm a mesma estrutura e as barreiras são iguais. Com *compute*, o destino seria uma `STORAGE_IMAGE` no *layout* `GENERAL` e as barreiras usariam o estágio `COMPUTE_SHADER`.

Relação com a técnica. O desenho segue as quatro fases da Seção 2.2: *bright pass* (passe 1), *downsampling* (passes 2 a 5), *upsampling* com soma (passes 6 a 9) e composição com *tone mapping* (passe 10). Os recursos também são próprios do *Bloom*: a `sceneHDR` em ponto flutuante, a pirâmide `bloomMip`, o *blend* aditivo e o UBO com o limiar e a intensidade.

8 Pseudocódigo dos Shaders

Nesta entrega escrevemos o pseudocódigo dos sete *shaders* do pipeline da Entrega 2 (Seção 7). Os arquivos estão na pasta `shaders/` do `.zip`: dois `.vert` e cinco `.frag`. Não usamos *compute shader*, então não há `.comp`. A sintaxe é a do GLSL do Vulkan (`#version 450`), mas o código ainda não foi ligado a uma aplicação. Nos comentários, "eq. 1" a "eq. 5" são as equações da Seção 3.

8.1 Objetivo de Cada Shader

Tabela 4: Os *shaders* da entrega.

Arquivo	Tipo	Passe	O que faz
`scene.vert`	*vertex*	0	transforma os vértices da cena para a tela
`scene.frag`	*fragment*	0	calcula a cor da cena em HDR (sem limitar em 1.0)
`fullscreen.vert`	*vertex*	1 a 10	gera o triângulo de tela cheia dos passes de pós-processamento
`bright.frag`	*fragment*	1	deixa passar só as partes claras da cena (*bright pass*)
`downsample.frag`	*fragment*	2 a 5	reduz a imagem pela metade e borra
`upsample.frag`	*fragment*	6 a 9	amplia o nível menor e soma no maior
`composite.frag`	*fragment*	10	soma cena e *bloom* e faz o *tone mapping*

8.2 Fluxo entre os Shaders

O fluxo é o da Figura 3: o *fragment shader* de um passe escreve em uma imagem e o do passe seguinte lê essa imagem como textura, depois da barreira correspondente (Seção 7.6).

`scene.vert` → `scene.frag` → `sceneHDR` → `bright.frag` → `bloomMip[0]` → `downsample.frag` (4 vezes) → `bloomMip[4]` → `upsample.frag` (4 vezes) → `bloomMip[0]` → `composite.frag` → *swapchain*

Nos passes 1 a 10 o *vertex shader* é sempre o `fullscreen.vert`. O `downsample.frag` e o `upsample.frag` rodam quatro vezes cada um, trocando a textura lida e os *push constants*.

8.3 Pseudocódigo Completo

Código 1: `scene.vert`

```
// scene.vert - vertex shader do passe 0 (cena)
// entrada: vertex buffer + UBO da camera / saida: gl_Position, normal e uv
#version 450

layout(location = 0) in vec3 inPosition;   // posicao do vertice
layout(location = 1) in vec3 inNormal;     // normal do vertice
layout(location = 2) in vec2 inTexCoord;   // coordenada de textura

layout(set = 0, binding = 0) uniform CameraUBO {   // std140, 192 bytes
    mat4 model;   // objeto -> mundo
    mat4 view;    // mundo -> camera
    mat4 proj;    // camera -> tela
} ubo;

layout(location = 0) out vec3 fragNormal;    // vai para o scene.frag
layout(location = 1) out vec2 fragTexCoord;

void main() {
    // posicao na tela (o hardware divide por w e usa z como profundidade)
    gl_Position = ubo.proj * ubo.view * ubo.model * vec4(inPosition, 1.0);

    // normal so gira (parte 3x3 da model) e volta a ter tamanho 1
    fragNormal = normalize(mat3(ubo.model) * inNormal);

    fragTexCoord = inTexCoord;   // passa direto
}
```

Código 2: `scene.frag`

```
// scene.frag - fragment shader do passe 0 (cena)
// entrada: normal, uv e textura do material / saida: cor HDR na sceneHDR
#version 450

layout(location = 0) in vec3 fragNormal;
layout(location = 1) in vec2 fragTexCoord;

// textura do material: sampler trilinear (com mipmaps), REPEAT.
// o canal alfa marca o que emite luz (1 = emite)
layout(set = 0, binding = 1) uniform sampler2D texSampler;

layout(location = 0) out vec4 outColor;   // sceneHDR, pode passar de 1.0

const vec3 luzDir = normalize(vec3(0.3, 1.0, 0.5));   // direcao para a luz
const vec3 luzCor = vec3(4.0, 3.8, 3.5);               // luz forte, em HDR
```

```
const float ambiente = 0.05;
const float emissivo = 10.0;

void main() {
    vec4 texel = texture(texSampler, fragTexCoord);   // nivel do mipmap automatico
    vec3 albedo = texel.rgb;

    vec3 n = normalize(fragNormal);   // a interpolacao muda o tamanho

    // difusa de Lambert; max evita valor negativo nas faces de costas
    float difusa = max(dot(n, luzDir), 0.0);

    vec3 cor = albedo * ambiente + albedo * luzCor * difusa;
    cor += albedo * texel.a * emissivo;   // partes que emitem luz

    outColor = vec4(cor, 1.0);   // sem clamp, e HDR
}
```

Código 3: `fullscreen.vert`

```
// fullscreen.vert - vertex shader dos passes 1 a 10
// sem vertex buffer: 3 vertices fixos, escolhidos pelo gl_VertexIndex
#version 450

layout(location = 0) out vec2 outUV;   // uv para o fragment shader

// triangulo maior que a tela, o hardware corta o que sobra
vec2 positions[3] = vec2[](
    vec2(-1.0, -1.0),
    vec2( 3.0, -1.0),
    vec2(-1.0,  3.0)
);

void main() {
    vec2 pos = positions[gl_VertexIndex];
    gl_Position = vec4(pos, 0.0, 1.0);   // z = 0, nao tem teste de profundidade
    outUV = pos * 0.5 + 0.5;             // de [-1,1] para [0,1], sem inverter o y
}
```

Código 4: `bright.frag`

```
// bright.frag - fragment shader do passe 1 (bright pass)
// entrada: sceneHDR + UBO BloomParams / saida: bloomMip[0] (960x540)
#version 450

layout(location = 0) in vec2 inUV;

layout(set = 0, binding = 0) uniform BloomParams {   // std140
    float threshold;   // T da eq. 2
    float knee;        // k da eq. 2
    float intensity;   // usado so no composite
    vec2 texelSize;    // (1/1920, 1/1080)
} params;

layout(set = 0, binding = 1) uniform sampler2D sceneHDR;   // linear, clamp to edge

layout(location = 0) out vec4 outColor;

```

```
const float EPS = 0.0001;   // epsilon da eq. 2

void main() {
    // cada pixel daqui = bloco 2x2 da cena, entao faz a media dos 4
    vec2 offsets[4] = vec2[](vec2(-0.5, -0.5), vec2(0.5, -0.5),
                             vec2(-0.5,  0.5), vec2(0.5,  0.5));
    vec3 cor = vec3(0.0);
    for (int i = 0; i < 4; i++) {   // 4 vezes, uma por pixel do bloco
        cor += texture(sceneHDR, inUV + offsets[i] * params.texelSize).rgb;
    }
    cor = cor / 4.0;
    cor = max(cor, vec3(0.0));   // nao existe luz negativa

    // luminancia (eq. 1)
    float lum = dot(cor, vec3(0.2126, 0.7152, 0.0722));

    // soft knee (eq. 2): peso vai de 0 (em T - k) ate k (em T + k)
    float T = params.threshold;
    float k = params.knee;
    float x = clamp(lum - T + k, 0.0, 2.0 * k);
    float peso = (x * x) / (4.0 * k + EPS);

    outColor = vec4(cor * peso, 1.0);   // so o que e claro sobra
}
```

Código 5: downsample.frag

```
// downsample.frag - fragment shader dos passes 2 a 5
// entrada: bloomMip[i-1] + push constants / saida: bloomMip[i] (metade do tamanho)
// filtro de 13 amostras (LearnOpenGL), faz o papel da gaussiana (eq. 4)
#version 450

layout(location = 0) in vec2 inUV;

layout(push_constant) uniform PushConstants {
    vec2 texelSize;   // 1 pixel da imagem lida
    int mipLevel;     // nivel escrito (1 a 4)
} pc;

layout(set = 0, binding = 0) uniform sampler2D srcTexture;   // linear, clamp to edge

layout(location = 0) out vec4 outColor;

void main() {
    float x = pc.texelSize.x;
    float y = pc.texelSize.y;

    // 13 amostras:   a - b - c
    //                - j - k -
    //                d - e - f    (e = pixel atual)
    //                - l - m -
    //                g - h - i
    vec3 a = texture(srcTexture, vec2(inUV.x - 2*x, inUV.y + 2*y)).rgb;
    vec3 b = texture(srcTexture, vec2(inUV.x,       inUV.y + 2*y)).rgb;
    vec3 c = texture(srcTexture, vec2(inUV.x + 2*x, inUV.y + 2*y)).rgb;

    vec3 d = texture(srcTexture, vec2(inUV.x - 2*x, inUV.y)).rgb;
    vec3 e = texture(srcTexture, vec2(inUV.x,       inUV.y)).rgb;
```

```
    vec3 f = texture(srcTexture, vec2(inUV.x + 2*x, inUV.y)).rgb;

    vec3 g = texture(srcTexture, vec2(inUV.x - 2*x, inUV.y - 2*y)).rgb;
    vec3 h = texture(srcTexture, vec2(inUV.x,       inUV.y - 2*y)).rgb;
    vec3 i = texture(srcTexture, vec2(inUV.x + 2*x, inUV.y - 2*y)).rgb;

    vec3 j = texture(srcTexture, vec2(inUV.x - x, inUV.y + y)).rgb;
    vec3 k = texture(srcTexture, vec2(inUV.x + x, inUV.y + y)).rgb;
    vec3 l = texture(srcTexture, vec2(inUV.x - x, inUV.y - y)).rgb;
    vec3 m = texture(srcTexture, vec2(inUV.x + x, inUV.y - y)).rgb;

    // media ponderada, os pesos somam 1
    vec3 cor = e * 0.125;
    cor += (a + c + g + i) * 0.03125;
    cor += (b + d + f + h) * 0.0625;
    cor += (j + k + l + m) * 0.125;

    // so no primeiro nivel: corta pixels muito fortes (ficam piscando)
    if (pc.mipLevel == 1) {
        cor = min(cor, vec3(10.0));
    }

    outColor = vec4(cor, 1.0);
}
```

Código 6: upsample.frag

```
// upsample.frag - fragment shader dos passes 6 a 9
// entrada: bloomMip[i+1] + push constants / saida: bloomMip[i]
// a soma com o que ja esta em bloomMip[i] e feita pelo blend aditivo (ONE + ONE)
#version 450

layout(location = 0) in vec2 inUV;

layout(push_constant) uniform PushConstants {
    vec2 texelSize;   // 1 pixel da imagem lida (a menor)
    int mipLevel;     // nivel que recebe a soma (3 a 0), so para debug
} pc;

layout(set = 0, binding = 0) uniform sampler2D srcTexture;   // linear, clamp to edge

layout(location = 0) out vec4 outColor;

float pesos[3] = float[](1.0, 2.0, 1.0);   // filtro tenda, /16 no total

void main() {
    vec3 cor = vec3(0.0);

    // 3x3 = 9 amostras, deslocadas de 1 pixel da imagem menor
    for (int y = -1; y <= 1; y++) {
        for (int x = -1; x <= 1; x++) {
            float peso = pesos[x + 1] * pesos[y + 1] / 16.0;
            vec2 uv = inUV + vec2(x, y) * pc.texelSize;
            cor += texture(srcTexture, uv).rgb * peso;
        }
    }

    outColor = vec4(cor, 1.0);   // o blend soma no bloomMip[i]
```

```
}
```

Código 7: `composite.frag`

```
// composite.frag - fragment shader do passe 10 (cena + bloom + tone mapping)
// entrada: sceneHDR, bloomMip[0] e UBO BloomParams / saida: imagem do swapchain
#version 450

layout(location = 0) in vec2 inUV;

layout(set = 0, binding = 0) uniform BloomParams {
    float threshold;
    float knee;
    float intensity;   // alfa da eq. 5
    vec2 texelSize;
} params;

layout(set = 0, binding = 1) uniform sampler2D sceneHDR;   // 1920x1080
layout(set = 0, binding = 2) uniform sampler2D bloomTex;   // bloomMip[0], 960x540

layout(location = 0) out vec4 outColor;   // swapchain (B8G8R8A8_SRGB)

void main() {
    vec3 cena = texture(sceneHDR, inUV).rgb;
    vec3 bloom = texture(bloomTex, inUV).rgb;   // o filtro linear amplia o halo

    vec3 total = cena + params.intensity * bloom;   // C_total da eq. 5

    vec3 cor = total / (1.0 + total);   // Reinhard, fica entre 0 e 1

    outColor = vec4(clamp(cor, 0.0, 1.0), 1.0);   // gama e feita pelo formato _SRGB
}
```

8.4 Entradas e Saídas

- `scene.vert`: entram posição, normal e uv do *vertex buffer* e as matrizes do UBO da câmera; saem `gl_Position`, normal e uv.
- `scene.frag`: entram normal, uv e a textura do material; sai a cor HDR na `sceneHDR`.
- `fullscreen.vert`: entra só o `gl_VertexIndex`; saem `gl_Position` e o `uv`.
- `bright.frag`: entram `uv`, `sceneHDR` e o UBO `BloomParams`; sai a `bloomMip[0]`.
- `downsample.frag` e `upsample.frag`: entram `uv`, o nível lido e os *push constants* (`texelSize` e `mipLevel`); sai o nível escrito (no *upsample*, somado pelo *blend*).
- `composite.frag`: entram `uv`, `sceneHDR`, `bloomMip[0]` e o `intensity`; sai a imagem final no *swapchain*.

Os tipos (`vec2`, `vec3`, `mat4`, `sampler2D`) e os *bindings* estão nas declarações de cada arquivo.

8.5 Equações e Variáveis

- `scene.vert`: posição $= \mathbf{P}\,\mathbf{V}\,\mathbf{M}\,p$; a normal usa só a parte 3×3 da `model` e é normalizada de novo.
- `scene.frag`: difusa de Lambert, $\max(\mathbf{n} \cdot \mathbf{l}, 0)$, com luz de intensidade 4.0 e emissivo 10.0. Por isso a cena passa de 1.0.
- `bright.frag`: `lum` é a Equação 1 e `peso` é a Equação 2, com `T` = `threshold`, `k` = `knee` e `EPS` = ϵ. O peso vale 0 até $L = T - k$ e fica em k a partir de $L = T + k$.
- `downsample.frag`: 13 amostras com pesos 1/8 (centro e internas), 1/16 (bordas) e 1/32 (cantos), que somam 1.
- `upsample.frag`: filtro tenda $\frac{1}{16}\begin{bmatrix}1 & 2 & 1\\ 2 & 4 & 2\\ 1 & 2 & 1\end{bmatrix}$, que também soma 1. Os dois filtros fazem o papel da Gaussiana da Equação 4: o borrão fica largo porque a imagem cai pela metade a cada nível.
- `composite.frag`: Equação 5, `total` $= C_{\text{cena}} + \alpha\, C_{\text{bloom}}$ e `cor` = `total`$/(1 +$ `total`$)$. Uma luz de 10.0 vira 0.91 e uma parede branca de 1.0 vira 0.5.

8.6 Loops e Condições

O `bright.frag` tem um `for` de 4 iterações (uma por pixel do bloco 2×2) e o `upsample.frag` tem dois `for` aninhados de -1 a 1 (9 amostras). No `downsample.frag` as 13 amostras estão escritas uma a uma, e o `if (pc.mipLevel == 1)` corta valores acima de 10.0 só no primeiro nível, para pixels isolados não piscarem. Casos extremos: o `max` zera luz negativa (`scene.frag` e `bright.frag`), o `EPS` evita divisão por zero, o `clamp` do `composite.frag` garante o intervalo do *swapchain* e o `CLAMP_TO_EDGE` do *sampler* repete o pixel da borda em vez de ler fora da imagem.

8.7 Amostragem de Texturas

Todas as leituras usam `sampler2D` e `texture()`. A textura do material tem mipmaps e *sampler* trilinear com `REPEAT`; o nível é escolhido pelo hardware. As imagens do *bloom* têm um nível só (o `texture()` lê o nível 0), filtro `LINEAR` e `CLAMP_TO_EDGE`. O filtro linear faz a média dos vizinhos quando a amostra cai entre dois pixels (`bright` e `downsample`) e amplia a imagem menor sem serrilhado (`upsample` e `composite`). Os deslocamentos são múltiplos do `texelSize` da imagem lida.

8.8 Resultado Esperado

A `bloomMip[0]` sai do *bright pass* quase toda preta, só com as luzes. Cada *downsample* gera um nível menor e mais borrado, até 60×33, e cada *upsample* soma o nível menor no maior; no fim a `bloomMip[0]` tem um halo largo e suave em volta de cada luz. O `composite.frag` soma esse halo na cena e comprime para $[0, 1]$: é a imagem final, com as luzes "sangrando" como nas Figuras 1 e 2.

8.9 Relação com a Técnica

Os *shaders* são as quatro fases da Seção 2.2: *bright pass* (Equações 1 e 2), *downsampling*, *upsampling* com soma e *tone mapping* (Equação 5). A pirâmide substitui a Gaussiana grande da Equação 4, e a soma de borrões de vários tamanhos aproxima o espalhamento da Equação 3 [2]. Cada arquivo corresponde a um passe e a um *descriptor set* da Seção 7.

9 Conclusão

O efeito de *Bloom* consolidou-se como um pilar indispensável da renderização computacional em tempo real. Sua fundamentação em fenômenos físicos e fisiológicos de dispersão intraocular, aliada às capacidades de baixo nível da API Vulkan — como controle explícito de barreiras de sincronização e processamento massivo via *Compute Shaders* —, viabiliza a emulação convincente de faixas dinâmicas extremas com baixo impacto de desempenho.

Referências

[1] H. Seetzen, W. Heidrich, W. Stuerzlinger, G. Ward, L. Whitehead, M. Trentacoste, A. Ghosh, and A. Vorozcovs, "High dynamic range display systems," *ACM Transactions on Graphics (TOG)*, vol. 23, no. 3, pp. 760–768, 2004.

[2] G. Spencer, P. Shirley, K. Zimmerman, and D. P. Greenberg, "Physically-based glare effects for digital images," in *Proceedings of the 22nd Annual Conference on Computer Graphics and Interactive Techniques (ACM SIGGRAPH)*, pp. 325–334, 1995.

[3] E. Reinhard, M. Stark, P. Shirley, and J. Ferwerda, "Photographic tone reproduction for digital images," *ACM Transactions on Graphics (TOG)*, vol. 21, no. 3, pp. 267–276, 2002.

[4] A. Ignatenko and A. Barladyan, "Using quad-trees for acceleration of physically-based image-space rendering of glare," in *Proceedings of GraphiCon'2008 (International Conference on Computer Graphics and Vision)*, pp. 128–133, 2008.

[5] M. Čadík, M. Wimmer, L. Neumann, and P. Ďurikovivč, "The naturalness of reproduced high dynamic range images," in *Ninth International Conference on Information Visualisation (IV'05)*, pp. 569–576, IEEE, 2005.